目录

第 1 章 AI Agent 入门 **1**

1.1 现代 Agent = LLM + 上下文 + 工具 1

1.1.1 观察空间与动作空间：模型与世界的接口 2

1.1.2 工具：Agent 的手脚 3

1.1.3 LLM：Agent 的大脑 5

1.1.4 上下文：Agent 的眼睛 6

1.1.5 ReAct 循环 8

1.2 Harness 工程：模型之外的竞争力 11

1.2.1 从提示工程到 Loop 工程：工程范式的演进 13

1.2.2 构建有效 Agent 的核心原则 13

1.2.3 如何选择模型 14

1.2.4 编排模式：工作流与自主 14

1.2.5 护栏与安全性 18

1.2.6 Harness 五要素与「构建」部分的对应 20

1.3 贯穿全书的设计模式 20

1.4 本章小结 21

思考题 22

第 1 章 AI Agent 入门

如果你用 Cursor 写过代码，看它搜索代码库、编辑多个文件、运行测试直到通过；用 Deep Research 调研过一个课题，看它反复搜索、阅读，总结出一份完整报告；用 Manus 操控浏览器帮你完成在线任务；让豆包手机助手帮你在手机上订票、发消息；或者让 Pine AI 替你打电话给运营商协商降低账单——你已经在使用 AI Agent 了。

这些产品的形态各异，但有一个共同点：它们不再是“你问一句、它答一句”的被动对话，而是能够自主规划执行步骤、调用各种工具完成任务，并根据结果不断调整策略的智能系统。AI Agent 正在成为我们与计算机交互的一种全新方式。

本章将带你从实践出发理解 AI Agent 的核心组成。我们将直接动手体验现代 Agent 的能力，理解其背后的架构原理，掌握构建 Agent 系统的设计模式与最佳实践。

> **阅读提示**：本章是全书的概念地图——它会快速引入 Agent 的核心公式、运行循环、工程框架和设计模式，为后续章节提供统一的术语和参照坐标。初次阅读时不必逐一记住所有概念，建议先建立整体印象；后续每一章都会展开讲解本章提到的某一个方面，届时可随时回来对照。

1.1 现代 Agent = LLM + 上下文 + 工具

现代 Agent 的最小工程实现可以用一个简洁的公式来表达：**Agent = LLM（大语言模型，Large Language Model）+ 上下文 + 工具**。这里的加号表示工程组件的组合，而不是强化学习中的形式化定义；更重要的是，这个公式只描述 Agent 边界之内的实现，**不包含 Agent 与之交互的 Environment（环境）**。其中每个词都需要做广义但边界清楚的理解：

- **LLM 是 Agent 的大脑**：它不只是一组模型参数，而是 Agent 的整个决策内核——理解意图、思考规划、做出判断。就像人类大脑不只是神经元的集合，还包括通过经验塑造的思维方式，LLM 的能力也来自两部分：**预训练**所积累的世界知识与语言能力，以及**后训练**所固化的决策策略——后者的具体技术（如监督微调与强化学习）将在第八章展开。
- **上下文是 Agent 的眼睛**：它不只是输入给模型的那段文本，而是 Agent 在每个决策点收到并保留的信息表示——来自环境的观察、用户记忆、领域知识、自身状态和任务进展。
- **工具是 Agent 的手脚**：这里的“工具”指 Agent 用来感知或改变外部世界的接口，包括工具定义、调用协议和适配器——从预定义的工具调用到动态生成代码，从委托子 Agent 协作到主动与用户沟通。

换一种更直观的说法：**Agent = 大脑 + 眼睛 + 手脚**。大脑负责思考和决策，眼睛接收环境提供的观察，手脚将决策转化为作用于环境的行动。

在经典的强化学习和控制论视角下，Agent 与 Environment 是闭环交互的两方，而不是彼此的组成部分。环境不断向 Agent 返回当前观察，Agent 根据已有上下文选择下一步行动；行动改变环境状态，新的状态再产生下一次观察，循环由此继续。这是理解所有 Agent 交互的最小结构。

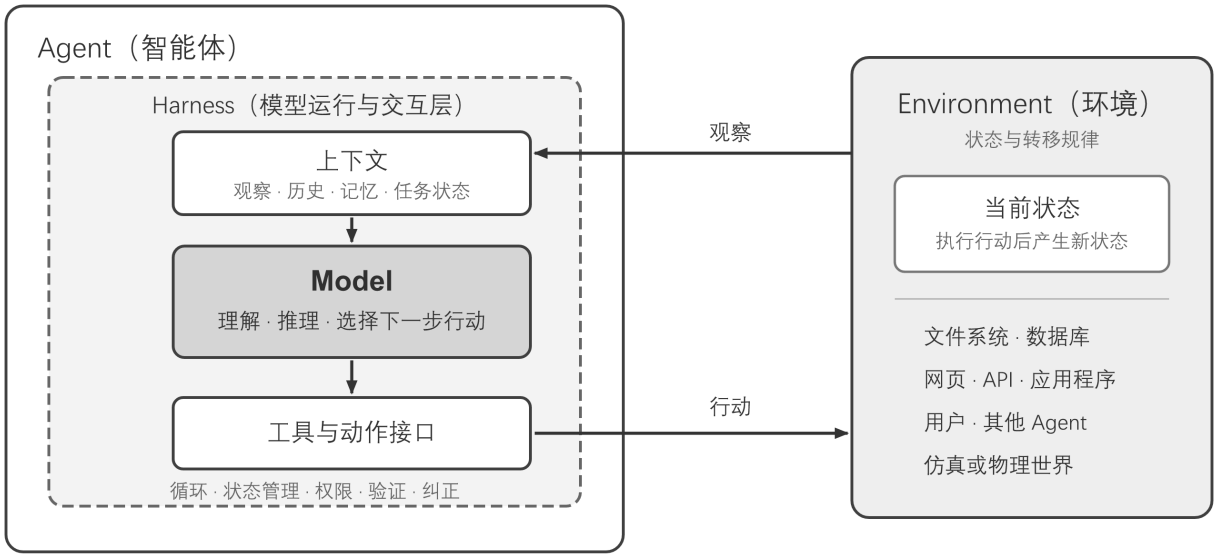


图 1-1 Agent 与 Environment 的闭环交互，以及 Agent 内部的 Model–Harness 结构

图 1-1 同时给出了两个抽象层次。外层是 **Agent 与 Environment 的交互关系**：环境包含文件、数据库、网页、用户、其他 Agent 以及物理或仿真世界，Agent 只能通过观察和行动接口与它交互。内层是 **Agent 的 Model–Harness 结构**：Model 负责策略决策；Harness 是 Agent 边界内环绕模型的运行与治理层，负责构造上下文、暴露工具接口、维护循环和状态，并实施权限、验证与纠正。Harness 可以创建、隔离或代理一个环境，却不因此包含环境自身的状态与转移规律。

本节开头的工程公式可以据此重新展开：LLM 对应 Model，“上下文 + 工具”构成最小 Harness；生产系统还会在 Harness 中加入约束、验证和纠正。后文所有架构都遵循这条边界。

这三个工程组件可以映射到 RL（强化学习，详见第八章）的策略与交互接口，但不是严格的一一等同关系：上下文是具体观察和历史在 Agent 内部的表示，并不等于整个观察空间；工具则定义 Agent 可使用的观察与行动接口，工具背后的对象仍属于环境。

直觉理解	实现组件	学术概念	含义
大脑	LLM	**策略** (Policy)	Agent 决定“下一步做什么”的决策逻辑——面对当前看到的信息，从所有可选行动中挑出最合适的一个
眼睛	上下文构造	**观察与历史**	将环境返回的观察与已有历史组织成当前决策所需的信息
手脚	工具与适配器	**观察/行动接口**	规定 Agent 可以读取哪些观察、发出哪些行动，以及接口采用什么格式

1.1.1 观察空间与动作空间：模型与世界的接口

观察通道与动作接口共同构成了 Agent 与外部环境之间的边界。Harness 把环境返回的观察转换为模型能够处理的上下文，再把模型选择的行动转换为对环境的工具调用。没有通过观察通道进入上下文的信息，对模型来说就像不存在；没有被动作接口允许的操作，模型即使知道该怎么做，也只能停留在文字建议上。

因此，**在底层模型固定时，提升 Agent 任务表现最主要的系统工程手段，往往就是重新定义或扩展观察空间与动作空间。**用本书的术语说，就是扩展上下文和工具。许多看似需要“更聪明模型”的问题，其实只是接口问题：把任务所需的数据纳入上下文，或把完成任务所需的操作封装成工具，原本不可解的任务就可能变得可解。

Manus：合并原本分离的空间。在 Manus 出现之前，生产级 Agent 大多沿着 Deep Research（深度调研）、Coding（代码生成）和 Computer Use（电脑操控）三条相对独立的路线发展。Manus 的突破在于率先把三者放进同一个有广泛影响力的生产级 Agent：虚拟浏览器扩大了观察空间，文件系统、代码执行与命令行执行扩大了动作空间。它没有仅靠替换一个更强的模型来成为通用 Agent，而是取三类 Agent 观察空间与动作空间的并集，使同一个 Agent 能跨越原有产品边界完成任务。

OpenClaw：把接口延伸到用户的数字生活。OpenClaw 又把这两个空间向外推进了一层。它通过用户已经在使用的 WhatsApp、Telegram、Slack、Discord、iMessage 等消息渠道接收任务和返回结果，让 Agent 可以随时随地被触达；同时采用本地 Gateway，连接 Google Drive、Notion 等云应用以及本地文件系统。这样一来，分散在不同账号与设备中的数字文件都可以在用户明确授权后进入同一个 Agent 的观察空间，并被其工具处理。相较于早期 Manus 以隔离云端沙盒为中心、往往需要上传文件或另行配置连接器的形态，本地优先的 OpenClaw 跨越了更大的数据边界。值得注意的是，Manus 后来也加入了 Google Drive 连接器和桌面端本地访问，这恰好再次说明，产品能力的演进往往就是观察空间和动作空间的演进[1]。

理解这三者的作用及其相互关系，是构建有效 Agent 系统的基础。我们从最具体的手脚（工具）开始介绍，逐步深入到大脑（LLM）和眼睛（上下文）。先来看看不同类型的 Agent 如何在这三个维度上展开：

Agent 产品	眼睛（感知）	手脚（行动）	策略
Cursor 等 Coding Agent	需求、读取到的代码片段、目录列表、终端输出	开放式（代码搜索、文件读写、执行命令等）	增量开发：理解需求 → 搜索相关代码 → 编辑代码 → 测试验证 → 调试修复
Deep Research 等搜索 Agent	搜索结果、网页内容、论文摘要与引用	开放式（搜索查询、网页读取、生成报告等）	迭代深化：根据已有信息调整搜索方向，逐步综合出完整报告
Browser Use 等电脑操控 Agent	屏幕截图、DOM 或无障碍树、操作结果	开放式（点击、输入、滚动、截图、执行代码等）	视觉感知 + 操作：观察屏幕 → 识别目标元素 → 执行操作 → 验证结果
豆包等手机助手 Agent	手机截图、App 界面状态、系统反馈	开放式（点击、滑动、输入、打开 App 等）	意图理解 +App 操控：理解用户需求 → 定位目标 App→ 执行操作 → 确认完成
Pine AI 等个人办事 Agent	经授权读取的账户记录、账单结果、服务商资料	开放式（打电话、发邮件、填表单、与用户确认）	多步骤任务执行：收集信息 → 制定协商策略 → 联系服务商 → 谈判 → 汇报结果

这些 Agent 系统有几个共同特征：它们都使用**开放式的动作空间**——不是从有限的几个按钮中选择，而是能生成任意自然语言和代码；它们都能**内部思考**——在采取行动前先思考和规划；它们都能**持续交互**——根据环境反馈不断调整策略。这些能力正是来自大脑、眼睛和手脚——即 LLM、上下文和工具——的协同作用。

1.1.2 工具：Agent 的手脚

工具是 Agent 与外部世界交互的桥梁：感知类工具承载环境到 Agent 的观察，执行类工具承载 Agent 到环境的行动。没有工具，Agent 只能“纸上谈兵”；有了工具，它才能真正读取或改变世界。

为了系统化地讨论工具，可以根据 Agent 与外界互动的方向把工具分为五类。下面先快速过一遍每一类的代表场景，建立整体印象，后续章节会逐一展开。

[1] Manus 的官方资料将其原始 Sandbox 描述为隔离的云端虚拟机；后来发布 Google Drive Connector 时也明确回顾了此前需要在 Drive、桌面与 Manus 之间手工下载和上传文件的割裂流程。2026 年 3 月发布 My Computer 时，Manus 又把“重要工作位于本地而非云端”称为云沙盒的根本局限。OpenClaw 的官方 README 则将其描述为运行在用户自己设备上的本地优先、常驻个人助手，并列出二十余种消息渠道；其工具和插件机制可继续接入云服务与本地能力。参见 https://manus.im/blog/manus-sandbox、https://manus.im/blog/manus-google-drive-connector、https://manus.im/blog/manus-my-computer-desktop、https://github.com/openclaw/openclaw、https://docs.openclaw.ai/tools

感知工具让 Agent 能访问信息：搜索引擎提供实时网络数据，文件系统读取本地文档，API 和数据库则对接外部服务和企业核心数据。

执行工具让 Agent 改变世界：代码执行、文件操作、系统命令、外部 API 调用——决策由此变成实际行动。

协作工具让 Agent 与其他 Agent 分工合作：委托子 Agent 完成专项任务，在关键决策点请求人类确认，或在多 Agent 系统中协调行动。

事件触发工具与前三类在调用方式上有本质的区别——它们不是 Agent 主动调用的，而是作为外部输入来驱动 Agent 开始执行任务。比如收到一封新邮件、到了某个预定时间点、或另一个系统发出了 Webhook 回调，这些事件会激活 Agent，让它开始后续的思考和行动。事件适配器同样是 Environment 向 Agent 提供观察的通道，因此本书把它归入广义的工具体系。

用户沟通工具是 Agent 主动与用户建立连接、传递信息的渠道。与执行工具改变外部世界不同，用户沟通工具专注于信息的传递和交互——通过文字消息、语音通话、邮件等方式，将 Agent 的执行进展或主动关怀传达给用户。

以上五类工具的完整分类体系和设计原则将在第四章展开讨论。工具设计的质量直接决定了 Agent 能走多远——接口定义不清晰，模型就会乱用工具；错误处理不到位，工具一旦调用失败，Agent 就可能陷入死锁；权限控制太宽泛，Agent 一旦出错，后果就难以挽回。MCP（Model Context Protocol，模型上下文协议）标准的推广，正在让工具接入变得更容易。

工具调用（Tool Calling，也称 Function Calling）是现代 LLM Agent 的一项核心能力，它让模型能够通过结构化的方式调用外部工具。这种能力将 LLM 从一个纯粹的文本生成器转变为能够执行实际操作的智能系统。本书后续统一使用“工具调用”这一术语。

工具调用的流程分为四步：首先，在上下文里告诉模型有哪些工具可用（包括名称、用途和参数）；然后，模型自主判断要不要调用工具、调用哪个、传什么参数；接着，工具执行完毕后，结果被追加到上下文中；最后，模型据此决定下一步行动。这个循环就是后文要介绍的 ReAct 的基础。

以一个查天气的场景为例，四步流程在 API 层面的简化表示如下：

```
第一步：声明工具                    第二步：模型决定调用
tools: [{                          assistant: {
  name: "get_weather",               tool_calls: [{
  parameters: {                        function: "get_weather",
    city: "string"                     arguments: {city: " 北京"}
  }                                  }]
}]                                 }

第三步：结果追加到上下文             第四步：模型基于结果回复
tool: {                            assistant: {
  tool_call_id: "call_1",            content: " 北京今天 28°C，晴。"
  content: '{"temp":28,"sky":" 晴"}'  }
}
```

开发者只需要定义工具和执行工具调用，模型自主完成“要不要调用、调哪个、传什么参数”的决策。第二章将详细展开这个 API 结构。

在为 Agent 设计工具时，可以先从任务所需的最窄能力起步，再随任务复杂度提升逐步扩展。如果任务只是做四则运算，一个参数清晰的计算器就足够了；当任务升级为读取表格、清洗缺失值、计算统计量并绘图时，受限的 Python 代码解释器就比不断叠加专用工具更容易组合和探索。但通用性也扩大了出错和攻击面：代码必须在隔离沙盒中运行，默认不能访问网络，也不能读取授权工作目录以外的文件，并对执行时间、CPU、内存和输出大小设置上限。

同样，单一日志工具适合记录一段执行过程；对于需要数小时甚至数天的长程任务，受控的虚拟工作目录则可以同时保存计划、中间

结果、运行日志和最终产物，让 Agent 能在多次执行之间接续工作。这个目录也应限定可读写的路径、容量和文件类型，并防止路径越界，而不是把宿主文件系统全部暴露给 Agent。

通用工具并不总是优于专用工具。支付、删除数据、发送邮件和生产部署等高风险或强业务约束操作，仍应封装为参数明确、权限受限且全程可审计的专用工具，必要时再加上预览和人工确认。因此，工具设计的核心原则是：**通用基础能力用于组合与探索；专用工具用于约束高风险和强业务规则操作。**

1.1.3 LLM：Agent 的大脑

大语言模型（Large Language Model, LLM）是 Agent 的决策核心。收到用户的请求后，它需要先解析真实意图（用户说的往往不是他真正想要的），再将模糊或复杂的任务拆解成可执行的步骤。执行过程中它还要持续做出判断：下一步该做什么、要不要调用工具、调哪个工具、传什么参数。这种“理解-规划-执行”的能力来自预训练所积累的知识，是工作流和自主 Agent 都依赖的基础。

LLM Agent 的一个独特能力是**内部思考**——在采取实际行动之前，Agent 可以先进行规划与推演。这一过程不改变外部环境，却能显著提升后续行动的质量。LLM 之所以能够进行有效的内部推演，得益于预训练（Pre-training，即在海量互联网文本上进行初始训练，让模型学会语言规律和世界知识）阶段习得的能力——模型在推演时所遵循的是人类知识中已经沉淀下来的逻辑规则，包括数学定律、因果关系、问题分解策略等。因此与传统强化学习 Agent 不同，今天基于 LLM 的 Agent 不是盲目的随机探索，而是在结构化的知识体系上展开。

1.1.3.1 模型即 Agent：当模型本身成为产品

“模型即 Agent”（Model as Agent）这一新范式代表了 AI Agent 发展的最新方向。先进模型通过后训练（特别是强化学习）将工具调用能力内化为原生能力：何时调用工具、调哪个、传什么参数，都由模型自己决定，无需人工编排。但这并不意味着框架层变得不重要了。恰恰相反，模型越强大，围绕模型构建的 Harness 就越关键。Harness 这个词原指马具，即套在马身上的缰绳与挽具，不是为了限制马的奔跑能力，而是把这种力量引导到正确的方向上。换到 Agent 语境里，模型是那匹强大但不可预测的马，Harness 则是把它的能力引导成可靠任务执行的工程外壳。在 Agent 中，Harness 包括上下文管理、工具接口、安全约束、验证与纠正等基础设施（详见本章末节）。

模型自主决策的空间越大，出错时的影响面也越大，因此需要更精细的约束、验证和纠正机制来确保可靠性。模型厂商的真正优势不是“让框架变薄”，而是能对模型与外围 Harness 进行协同优化，持续迭代。

但这里悬着一个更深的问题：如果模型持续变强，今天这些 Harness 会不会最终被模型“吃掉”？Rich Sutton 在《苦涩的教训》(The Bitter Lesson) 中回顾了 AI 研究七十年间反复上演的一幕[2]：研究者一次次把自己对领域的理解编码进系统，短期见效，长期却总是输给能随算力与数据规模持续扩展的通用方法——搜索与学习。以此衡量，Harness 里的约束、验证与纠正，有多少属于“人类先验”，注定会被模型内化？本书的立场是：**方向认同，节奏务实。**方向上，本书不怀疑模型会持续吃掉 Harness——工具调用、长程规划都曾靠外部编排，如今已是模型的原生能力；但在节奏上，这个“吃”的过程远比想象中慢：训练以月计，模型也无法一次内化真实业务中所有的约束与偏好，模型此刻的能力边界，就是 Harness 此刻的价值所在。因此 Harness 工程不是对苦涩的教训的抵抗，而是这一教训在工程时间尺度上的实践：模型还做不稳的，Harness 先补上；模型每内化一层，Harness 就卸下一层，转而兜底新的能力前沿。

1.1.3.2 Agent 的学习机制：从上下文适应到持久更新

前面讨论了模型可以通过强化学习将工具调用策略内化为原生能力。但 Agent 的行为改变不只发生在训练阶段。按照更新发生的位置和持续时间，可以把它理解为三条互补路径（图 1-2）：任务内的上下文适应、跨任务的外部产物（artifact）更新，以及训练周期中的参数更新。

[2]Sutton, Rich. “The Bitter Lesson”, 2019. http://www.incompleteideas.net/IncIdeas/BitterLesson.html



图 1-2 Agent 能力更新的三个层次

上下文适应发生在当前任务中。示例、状态和检索结果进入上下文后，模型可以立即调整行为，却不会因此改变下一次会话的持久状态。它的优势是快速、低成本，局限是受上下文窗口和信息组织方式约束；第二章将详细讨论这种适应如何工作。

要让变化跨越任务保留下来，可以更新**外部产物**：把事实和经验整理为知识文档，把可语言化的策略写进 Prompt 或 Skill，把确定性流程与约束写成程序和 Harness。这些产物可审计、可修订，执行时仍需通过上下文或工具接口被 Agent 使用。第三至五章分别给出知识与程序基础，第九章则讨论如何从已评价的运行轨迹中生成这些更新。

当目标是医疗影像理解、自然语言风格或隐式决策策略等高维能力时，外部规则难以完整表达，就需要通过后训练更新**模型参数**。参数更新的部署成本较高，却能形成自然、广泛的泛化能力；第八章将系统介绍其方法。三条路径因此不是互斥分类，而是不同时间尺度上的协同机制：上下文负责临场适应，外部产物负责可控积累，参数负责内化难以显式表达的能力。

1.1.4 上下文：Agent 的眼睛

上下文是 Agent 在每个决策点能看到的全部信息。就像一个人在做决策时需要看到桌上摊开的所有资料——任务说明、参考手册、之前的沟通记录、最新的数据——Agent 的上下文窗口就是它的“视野”。从 API 的视角看（详见第二章），每次调用 LLM 时的上下文由以下五个部分构成：

- **系统提示词** (System Prompt)：与用户每次输入的提示词不同，系统提示词由开发者编写，在整个对话过程中保持不变，相当于 Agent 的“岗位说明书”——定义它的身份、权限和行为准则。通过提示工程（Prompt Engineering）精心设计系统提示词，我们可以塑造 Agent 的工作方式。系统提示词中还会包含跨会话保存的**用户记忆**（用户偏好、历史行为、背景设定等个性化信息，详见第三章）和动态注入的环境状态。
- **工具定义** (Tool Definitions)：声明 Agent 可用工具的名称、功能描述和参数格式。没有工具定义，Agent 就无法识别和调用任何工具，但它并不会因此停下来——消融实验（实验 1-1）会说明这一点。工具定义与系统提示词一起构成对话中保持不变的**静态前缀**（这是基础模式；2026 年以来，生产框架中工具的完整 schema 也可以按需动态加载到上下文末尾而不破坏前缀，详见第二章工具定义一节和第四章）。
- **用户消息** (User Messages)：来自用户的输入。用户消息中还可能包含通过 RAG（检索增强生成，Retrieval-Augmented Generation，详见第三章）动态检索引入的**外部知识**——覆盖训练数据截止后的信息或私有领域知识。
- **模型回复** (Assistant Messages)：模型之前生成的回复，最多包含三个部分——思考过程（`reasoning`，即内部思考链，用于

保持思维的连贯性和决策的可解释性）、文本内容（`content`，即对用户的回复）和工具调用请求（`tool_calls`，即 Agent 采取行动的方式）。在一次具体的回复中，三者不一定同时出现：例如 Agent 决定调用工具时通常只有 `reasoning` + `tool_calls`，给出最终回答时通常只有 `reasoning` + `content`。

- **工具执行结果**（Tool Results）：Agent 框架执行工具后返回的结果。这些结果是 Agent 下一步思考的直接依据，也让它能够从执行结果中学习、避免重复犯错。

前两项（系统提示词 + 工具定义）是静态前缀，后三项（用户消息 + 模型回复 + 工具执行结果）是随交互不断增长的动态消息历史。这五个部分共同构成了 LLM 每次推理时的上下文。

要验证每个组件是否都不可或缺，最直接的方法是**消融实验**（Ablation Study）：就像医生诊断时逐一排除病因——先去掉 A 组件看系统是否还正常，再去掉 B 组件，以此类推，从而判断每个组件的贡献。实验 1-1 正是按这个思路，对上述五个组件做了系统性测试。

实验 1-1 ★★：上下文的关键作用

通过系统性的**消融实验**（Ablation Study），我们探索了不同上下文组件对 Agent 行为的影响。实验从上述五个部分中选取了四个组件进行测试——系统提示词作为 Agent 的基本身份定义不参与消融，因为没有系统提示词，Agent 连基本的角色认知都没有，测试没有意义。如图 1-3 所示，五组对照实验包括：一组保留全部组件的完整基线，再加上四组各缺失一个组件的对照，以此观察每个组件对 Agent 性能的影响。

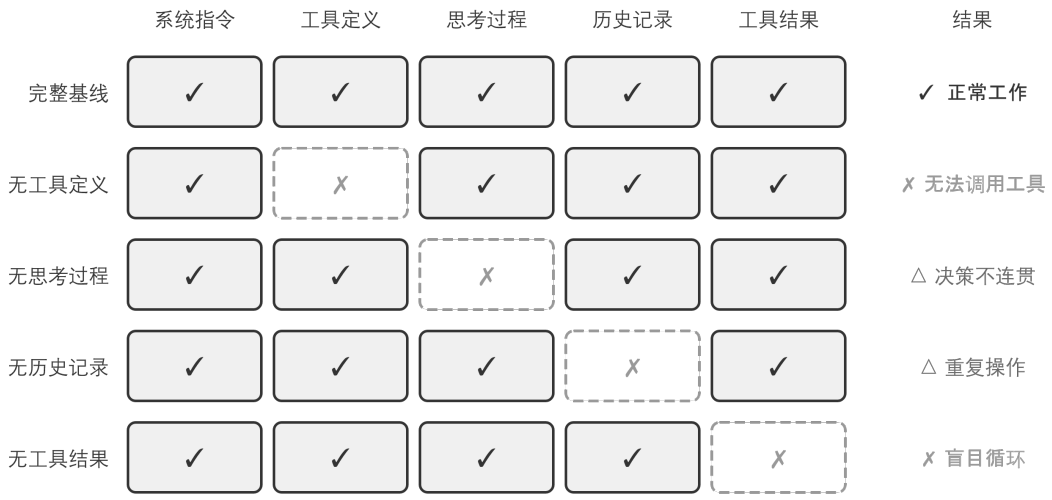


图 1-3 实验 1-1——上下文消融实验设计

实验结果揭示了每个上下文组件的作用，也揭示了它们并不同等重要。**工具定义**（Tool Definitions，静态前缀的一部分）是 Agent 行动能力的基础，没有它就无法调用任何工具；但失去行动能力不等于沉默——模型照样会给出一份格式工整、语气笃定的答案，数据却来自参数记忆，与真正基于观测得出的答案排版一模一样。它是坦白拒绝还是就地编造，主要取决于模型自身的幻觉率与诚实度；提示词里“不得自行估计”这类约束只能降低编造的概率，并不能消除。**工具执行结果**（Tool Results）是闭环控制的关键，缺失它会让 Agent “盲目”执行，反复重试直到耗尽迭代预算。**思考过程**（模型回复中的 reasoning 部分）记录的是“为什么这么做”，工具执行结果记录的是“发生了什么”；当前者可以从后者重建时，把它从历史中丢掉几乎没有代价。**历史消息**（之前轮次的用户消息、模型回复和工具执行结果）则防止了冗余操作，避免重复犯同样的错误。

这个实验的核心洞察是：**上下文决定了 Agent 能看到什么，而 Agent 只能基于它看到的信息做决策。**但组件并不等价，衡量的标准是它承载的信息能否从别处重建——“每个组件都不可或缺”这类论断要靠实测，而且模型迭代很快，同一个消融换到新模型上完全可能得出不同的结论。还有一点对工程实践更重要：**“给出了回答”不等于“完成了任务”**，上下文残缺时典型的失败不是报错退出，而是一个看上去毫无破绽的答案。

1.1.5 ReAct 循环

了解了 Agent 的三大组件后，一个自然的问题是：它们如何协同工作？ReAct 循环就是将 LLM、上下文和工具串联起来的核心机制——让我们看看一个 Agent 是如何一步步思考和行动的。

Agent 执行任务的核心模式叫做 **ReAct**（Reasoning + Acting）。虽然名字只体现了思考（Reasoning）和行动（Acting）两个词，但实际循环包含三个环节：模型先**思考**当前应该做什么，然后调用工具**行动**，再**观察**工具返回的结果并继续思考下一步。这个“想 → 做 → 看 → 想 → 做 → 看”的循环不断重复，直到任务完成。

让我们通过一个多币种收入汇总的具体例子来理解 Agent 的**轨迹**（trajectory）。轨迹是 Agent 在执行任务过程中不断积累的消息历史——用户消息、模型回复（包括思考过程和工具调用）、工具执行结果。每一次调用 LLM 时，它接收的完整上下文由**静态前缀**（系统提示词 + 工具定义）和**轨迹**（动态消息历史）两部分组成（图 1-4）。这揭示了一个关键事实：**Agent 的上下文 = 静态前缀 + 轨迹。**具体地说，静态前缀对应前文五个组件中的前两项（系统提示词 + 工具定义），轨迹对应后三项（用户消息 + 模型回复 + 工具执行结果，随交互不断增长）。基于这个完整上下文，LLM 生成下一步的响应，然后这个响应又追加到轨迹中，供下一次调用使用。

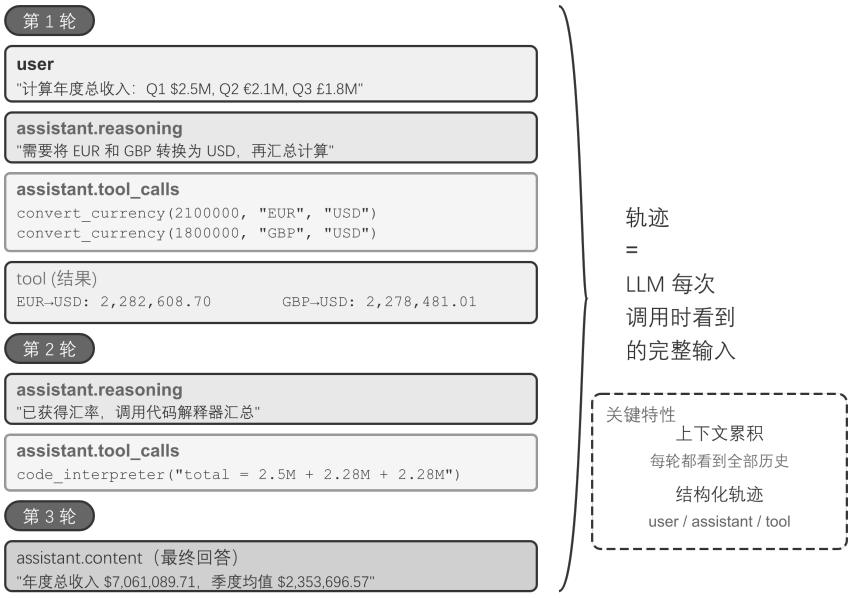


图 1-4 Agent 轨迹——多币种汇总任务的 ReAct 循环

先看最小运行骨架。它说明的是**机制如何运行**：Model 只负责决定下一步，Harness 负责组装上下文、校验并执行工具，Environment 负责产生真实状态变化和观察。本书后续也沿用 Python 风格伪代码；伪代码不能直接运行，也不对应某个 SDK。具体的可执行代码在本书配套代码仓库中。

```
trajectory = [user_request]

repeat:
    context = stable_prefix + trajectory
    decision = Model(context)
    trajectory.append(decision)

    if decision has no tool call:
```

```
        return decision.answer

    for call in decision.tool_calls:       # independent calls may run in parallel
        validated_call = Harness.validate(call)
        observation = Environment.execute(validated_call)
        trajectory.append(observation)
```

下面再看一次运行后**轨迹中保存了什么**。它是消息数据的结构示意，不是 Agent 循环的实现代码：

```
轨迹 = [
  {role: "user" , content: " 根据公司季度收入：Q1 2.5M 美元，Q2 2.1M 欧元，Q3 1.8M 英镑，Q4 380M 日元，计
↪   算公司年度总收入和季度平均收入" },

  # 第一次迭代 - LLM 看到上述轨迹，生成响应
  {role: "assistant" ,
   reasoning: " 需要将所有货币转换为 USD..." ,
   content: "" ,  # 没有直接回复用户
   tool_calls: [
     {name: "convert_currency" , args: {amount: 2100000, from: "EUR" , to: "USD" }},
     {name: "convert_currency" , args: {amount: 1800000, from: "GBP" , to: "USD" }},
     {name: "convert_currency" , args: {amount: 380000000, from: "JPY" , to: "USD" }}
   ]},

  # Agent 框架执行工具，添加结果到轨迹
  {role: "tool" , content: "EUR->USD: 2282608.7" },
  {role: "tool" , content: "GBP->USD: 2278481.01" },
  {role: "tool" , content: "JPY->USD: 2541806.02" },

  # 第二次迭代 - LLM 看到完整轨迹，包括工具结果
  {role: "assistant" ,
   reasoning: " 已获得转换结果，现在需要汇总计算..." ,
   content: "" ,
   tool_calls: [
     {name: "code_interpreter" , args: {code: "total = 2500000 + 2282608.7 + ..." }}
   ]},

  {role: "tool" , content: "Total: $9,602,895.73, Average: $2,400,723.93..." },

  # 第三次迭代 - LLM 看到完整轨迹，生成最终答案
  {role: "assistant" ,
   reasoning: " 所有计算完成，总结结果..." ,
   content: "FINAL ANSWER: 总收入 $9,602,895.73..." }
]
```

注意，轨迹中没有显示系统提示词和工具定义——它们作为静态前缀，在每次 LLM 调用时都会被自动拼接在轨迹前面。

整个过程只用了 3 次迭代、4 次工具调用。

在这种最基本的设计中，上下文是不断追加的：每次调用 LLM 都能看到完整的轨迹，因而它清楚任务进行到了哪一步、此前尝试过什么、得到了什么结果。轨迹的结构化也让系统易于解释和调试——用户消息、模型回复（思考过程 + 工具调用）和工具执行结果彼此分明。更进一步，分析大量轨迹可以发现 Agent 的行为模式、优化决策路径、改进工具设计；轨迹还可以沉淀进知识库，或用于强化学习训练更好的模型，形成从经验中学习的闭环。

理解了 Agent 的运行循环后，让我们通过两个实验来感受不同模型如何驱动这个循环。

实验 1-2 ★：Kimi K3 原生 Agent 能力

这个实验展示了 **Kimi K3** 的原生 Agent 能力，体现了“模型即 Agent”的新范式。Kimi K3 是一个约 2.8 万亿参数的混合专家 (MoE, Mixture of Experts) 模型——可以把 MoE 想象成一个专家团队：面对不同类型的问题，系统会自动选择最合适的几位专家来作答，而不需要所有专家同时上阵，这样既保证了能力又提高了效率。它拥有 100 万 token 的上下文窗口、原生的视觉理解能力，以及始终开启的“思考模式”(thinking mode)；模型通过强化学习训练，将工具调用的**决策策略**内化为原生能力——何时调用工具、调用哪个、传什么参数都由模型自主决定，从而能够自主完成网络搜索等任务。

关键观察包括：模型自己决定何时搜索、搜索什么，展现了真正的自主性；它能根据搜索结果动态调整策略，自主判断信息是否充足。这里需要厘清一个常见的误解，关键在于分清两件事的归属。**强化学习写进参数的是决策**——何时该调用工具、调用哪个、传入什么参数、拿到结果后是否继续、如何把几十上百次调用串联成连贯的推理。**工具本身及其执行则由 Agent 框架（或 API 内置工具）提供**——`web_search`、`code_runner` 的真实实现、代码沙盒环境、调用的发起与结果回传，都在模型之外的基础设施里完成 (Kimi 通过名为 Formula 的服务端脚本引擎运行这些官方工具)。因此编排循环并没有消失，只是决策权交给了模型；至于循环本身跑在哪里，取决于接入方式。在实验 1-3 的 Responses API 路径中，搜索、代码执行和多轮编排整体在服务端闭环完成；而在实验 1-2 的 Kimi 路径中，Formula 只负责在服务端执行搜索本身，“调用模型 → 追加工具结果 → 再次调用”的 ReAct 循环仍由客户端代码驱动（见 `chapter1/web-search-agent/agent.py` 中的 `while` 循环）[a]。

Kimi K3 在 Agent 任务中的一个突出优势是**长链工具调用的稳定性**——它能够连续执行 200 ~ 300 次工具调用而保持思考的一致性，远超多数模型在数十次调用后就开始退化的表现。K3 面向长周期编程与 Agent 工作负载优化，发布时提供 K3 Max（面向对话与 Agent 任务）与 K3 Swarm Max（面向大规模并行处理）两个规格。作为开源模型，它在软件工程和 Agent 基准测试中展现了可与顶尖闭源系统比肩的性能，证明了通过强化学习赋予模型原生 Agent 能力这条路线的有效性。

[a] 感谢读者 asdlem 通过 GitHub Issue #30 指出并厘清了“RL 内化的是工具调用决策策略、而非工具执行机制”这一区分。参见 https://github.com/bojieli/ai-agent-book/issues/30

实验 1-3 ★：GPT-5.6 原生 Deep Research 能力

第二个实验使用 **OpenAI GPT-5.6**，展示先进模型如何借助 API 内置工具，在服务端形成 Deep Research 的“搜索—阅读—分析”编排闭环。GPT-5.6 的一个便利特性是**自由格式工具调用** (Freeform Tool Calling)。传统方式中，模型调用工具时必须把所有参数打包成严格的 JSON 格式（一种结构化的数据格式），这就像填表格一样有很多格式限制。自由格式工具调用（在 API 中通过 `type: "custom"` 的工具类型声明）允许模型直接向工具发送原始文本（比如一段 Python 代码、一条 SQL 查询），省去了 JSON 转义的麻烦。要说明的是，这是 API 参数格式的演进，而非模型架构的革新——客户端的工具调用循环（检测 `tool_calls` → 执行 → 回传结果）逻辑保持不变，改变的只是参数从 JSON 字符串变成了原始文本。

GPT-5.6 配合 Responses API 的**网络搜索和代码解释器**内置工具——这正是 Deep Research 的核心：模型能够自主搜索网络获取实时信息，并编写代码进行深度分析，实现“搜索 -> 阅读 -> 分析 -> 再搜索”的迭代研究过程。例如，面对“东盟 10 国首都之间，最近的一对首都距离多少”这样的问题，GPT-5.6 会自动搜索各国首都的地理坐标，然后编写 Python 代码计算所有首都对之间的大圆距离，最终找出最近的一对。又如“搜索最近一个月的比特币走势，做技术分析”任务中，它能从多个金融数据源获取实时价格数据，运用专业的技术分析库计算移动平均线、RSI、MACD 等技术指标，生成可视化图表并给出交易建议。

更重要的是，GPT-5.6 将 **OpenAI Deep Research** 产品的设计理念内化到了模型层面，引入了**意图澄清过程**。当用户提出研究需求后，GPT-5.6 不会立即动手执行，而是首先通过一系列问题来澄清用户的真实意图。以“搜索最近一个月的比特币走势，做技术分析”为例，它会先问：“你偏好使用哪个数据源？需要分析哪些技术指标？”通过这种交互式的意图澄清，GPT-5.6 能够生成更精准、更符合用户需求的研究报告。

GPT-5.6 是“模型即 Agent”概念的一个成熟实例：网络搜索、代码解释器作为 Responses API 的内置工具在服务端闭环执行，客户端不必再自行搭建“搜索—阅读—分析”的编排框架。而意图澄清的意义在于，它让“用户说了什么”和“用户真正想要什么”之间的差距，在任务执行之前就得到了弥合。

需要说明的是，这个实验并不绑定某一家厂商。没有 OpenAI 额度的读者完全可以用具备等价托管工具的提供商复现：例如阿里云百炼 qwen3.7-plus 的 Responses API 同样内置 `web_search` 与 `code_interpreter`；Kimi K3 的 Formula 托管搜索与 `code_runner` 也属于同类能力。

图 1-5 展示了“模型即 Agent”范式下原生工具调用的完整架构，以及 Kimi K3 / GPT-5.6 在实际任务中的 ReAct 执行过程。注意图中“客户端无需手写 ReAct 循环”只对 Responses API 路径成立：两条路径都由模型决定何时调用哪个工具、工具都在服务端执行，区别在于 Kimi 路径的循环仍由客户端驱动。

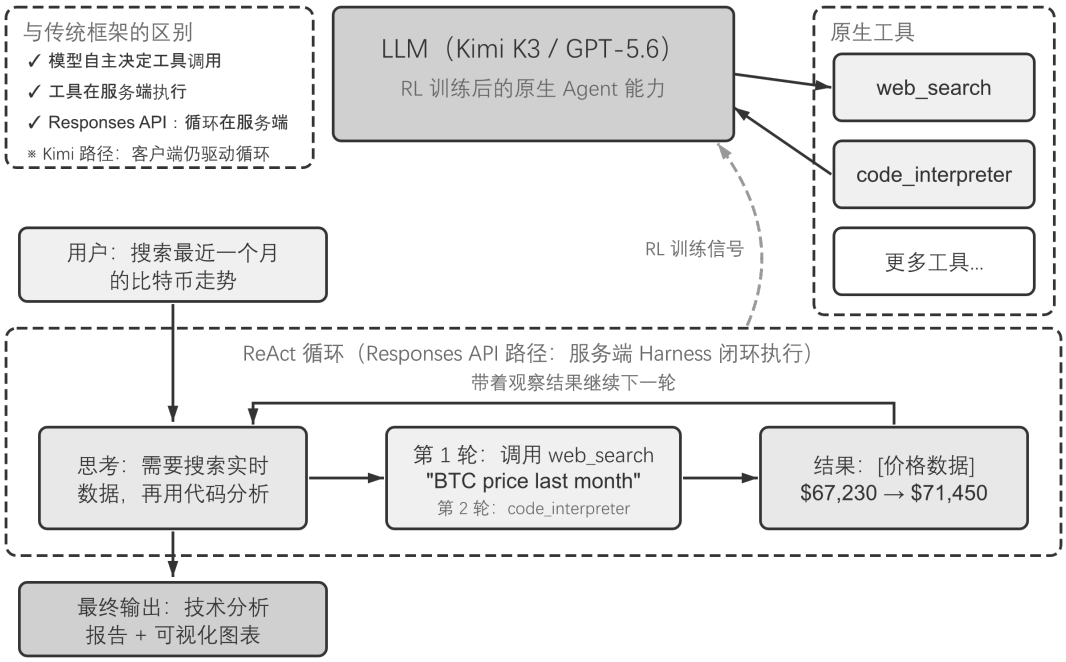


图 1-5 “模型即 Agent”架构——原生工具调用

1.2 Harness 工程：模型之外的竞争力

到这里你已经理解了 Agent 的核心工作原理——LLM 通过 ReAct 循环，在上下文的辅助下使用工具完成任务。前面的实验证明了这套基本机制是有效的，但同时也暴露了明显的脆弱点：模型可能产生幻觉（编造不存在的工具或参数）、选错工具，或在遇到错误时无法自我恢复。一个能跑的 Demo 和一个可靠的产品之间还有巨大的鸿沟，而这些脆弱点正是 Harness 工程要解决的问题。本章前半部分回答了 Agent 是什么，下半部分回答 Agent 如何在生产环境中可靠运行。

前面几节建立了 **Agent = LLM + 上下文 + 工具**的最小工程公式，并通过图 1-1 明确了 Agent 与 Environment 的边界。从 Harness 工程的视角看，可以把 LLM 抽象为核心组件 Model，把 Agent 边界内负责支撑模型运行及模型与环境交互的代码、配置和服务统称为 Harness。两个视角并非替代关系，而是不同抽象层次上对同一 Agent 实现的描述。Harness 的核心是原公式中的“上下文管理 + 工具接口”，再加上三层保障机制：**约束**（限定 Agent 能做什么、不能做什么）、**验证**（检查 Agent 做得对不对）和**纠正**（做错了怎么补救）。

用方程展开生产形态下的完整组成：

> **Agent = Model + Harness**
> **Harness = 上下文管理 + 工具接口 + 约束 + 验证 + 纠正**

Agent ↔ Environment

最小 Demo 只需 Model 和能构造上下文、暴露工具的 Harness；生产系统还要在同一边界内加入约束、验证和纠正。比如退款 Agent 可以把政策放进上下文、用权限和金额规则约束调用、用数据库状态验证结果，并在超时时重试或回退。Harness 工程研究的正是这层“模型之外、Agent 边界之内”的运行与治理代码。

更精确地说，**Harness 不是模型之外的一切**，而是 **Agent 边界内、模型之外**的运行与治理层。它负责协调 Model 与 Environment 的交互，但不包括与之交互的环境本身：工具定义、调用适配器、沙箱的权限与重置机制属于 Harness；沙箱内随行动变化的文件和进程、外部数据库、网页、用户及物理世界属于 Environment。物理部署位置也不能决定概念归属——即使仿真环境与 Agent 运行在同一进程中，它仍然是 Environment。Harness 的核心是上下文管理与工具接口，围绕它们构建了三类工程化保障机制：

功能	职责与核心原则	实际例子	详见
Context（上下文）	为模型提供感知信息；信息要充分，让 Agent 在每个决策点都基于足够的信息判断	系统提示词、知识库、Agent 状态栏、Sidecar 旁路查询	第二、三章
Tools（工具接口）	为模型提供观察与行动手段；接口要清晰，命名直观、参数有例子、边界有说明	MCP 工具、代码解释器、搜索工具	第四章
Constrain（约束）	设定行为边界；采用故障安全默认值，所有能力默认关闭，必须显式开放（类似手机 App 权限管理）	Claude Code 中每个工具默认需要用户授权才能执行	第四章
Verify（验证）	自动判断操作结果的对错；安全检查只看结构化数据（如工具返回的 JSON 字段），而不看模型自由生成的文本，因为后者可能已被提示注入操纵	Linter 检查、类型系统、工具调用结果校验	第五、六章
Correct（纠正）	发现问题时自动修正或回退；在确认无法恢复之前不暴露中间态，例如工具调用失败时先静默重试，不把半成品结果展示给用户	静默重试、接续生成、连续失败时回退到人工判断（熔断机制）	第二、五章

模型控制循环的基本流程如下伪代码所示：

```
observation = Environment.observe()
trajectory = [observation]
while true:
    actions = Model(Harness.build_context(trajectory))
    if len(actions) == 0:
        break
    allowed_actions = Harness.constrain(actions)
    observation = Environment.apply(allowed_actions)
    if not Harness.verify(Environment):
        observation = Harness.correct(Environment)
    trajectory.append(allowed_actions, observation)
```

这段骨架刻意不展开具体实现。完整的 API 消息循环将在第二章介绍，工具与自动验证则分别在第四、五章展开。

五个功能构成一个闭环：上下文与工具让 Agent “能做事”——理解任务并采取行动；约束预防错误，验证发现偏差，纠正使闭环得以形成，三者共同让 Agent “不做错事”。它们不是独立于上下文和工具之外的东西，而是确保上下文和工具在生产环境中可靠运转的工程实践。缺少任何一个环节，系统都会出现可靠性缺口。而在 Agent 产品的成熟度曲线上，两类功能的重要性是不对称的。

早期的 Agent 框架主要关注上下文与工具：给模型工具、给模型上下文，让它“能做事”。而生产级 Agent 系统的重心已经转向约束、验证与纠正：确保工具调用是安全的、上下文是经过管理的、错误是可纠正的。

以 Claude Code 为例，它的 Harness 中绝大部分代码都是约束、验证与纠正，而非上下文与工具——工具本身（文件读写、命令执行、搜索）只是一小部分，而围绕这些工具构建的保障机制才是真正的核心。这些机制包括：

- **流程状态管理**：追踪 Agent 当前执行到哪一步
- **多层上下文压缩**：当信息太多时自动精简
- **权限分类**：控制哪些操作需要用户确认
- **熔断器**（Circuit Breaker）：当错误连续发生时自动“断电”停止重试——就像家里电路短路时保险丝会自动跳闸，防止整个系统崩溃
- **错误恢复机制**：捕获异常、回滚到上一稳定状态、重试或交还给人类

行业正在从“能做事”向“可靠地做事”转变，Harness 工程因此成为 Agent 系统的核心竞争力。

1.2.1 从提示工程到 Loop 工程：工程范式的演进

回顾 AI 应用工程的发展，可以看到一条清晰的演进弧线：

提示工程（Prompt Engineering）是第一波创新——通过优化输入给模型的自然语言指令来提升输出质量。

上下文工程（Context Engineering）是第二波——人们认识到单纯优化提示词还不够，需要系统性地管理模型能看到的所有信息（系统指令、工具定义、对话历史、外部知识）。

Harness 工程是第三波——它将视野从“模型能看到什么”进一步扩展到“Agent 如何组织模型运行并与环境交互”，涵盖上下文与工具接口、约束机制、验证手段、反馈循环和错误恢复等 Agent 边界内、模型之外的运行与治理机制。

随后出现的 **Loop 工程**（Loop Engineering）又把视野从单次运行扩展到跨轮次的持续自主运转：谁来发现下一件该做的事、何时验证、何时才算真正完成（第十章将结合多 Agent 协作系统展开）。

2026 年 7 月，业界又开始用 **Graph 工程**（Graph Engineering）描述一种更高层的编排视角：把 Agent 循环、确定性程序和人工审批组织成显式的执行图，其中节点承担具体能力，边规定路由与依赖，结构化状态沿边传递并在关键边界处持久化[3]。

这五个阶段不是替代关系，而是层层包含的：提示工程是上下文工程的子集，上下文工程是 Harness 工程的子集，Harness 工程是 Loop 工程的子集，Loop 工程又是 Graph 工程的子集——单个 Agent 循环正是执行图中的一个节点。每一层都在前一层的基础上扩展了工程师的关注范围和影响力。**当各家模型的能力越来越接近、不再是决定性的差异因素时，竞争优势就转移到了模型之外的工程实践。**

这一判断在最近的工程实践中得到验证。LangChain 在 Terminal Bench 2.0（一个评估 Agent 在终端环境中完成复杂任务能力的基准测试）上的实践提供了一个有力例证：得分从 52.8% 提升到 66.5%（从排行榜 30 名开外跃升至前 5），改变的不是模型，而是 Harness，具体包括让 Agent 自动检查自己的执行结果、检测是否陷入重复循环、优化思考策略等工程手段。

1.2.2 构建有效 Agent 的核心原则

根据 Anthropic 的经验，成功的 Agent 系统遵循三个核心原则[4]。

保持简单。从最简单的方案开始，只在确实必要时才增加复杂度。直接的 API 调用优于复杂的框架，清晰的代码优于聪明的抽象。因为每多一层抽象都会成为以后调试时新的盲区。

[3] Josh C. Simmons 在 2026 年 7 月 4 日的文章 *We Are Entering the Graph Engineering Phase* 中较早明确使用这一名称，并将其概括为节点、类型化边和可检查点状态；7 月 18 日，Peter Steinberger 关于“是否已从 loops 转向 graphs”的讨论进一步推动了该名称传播。需要注意的是，相关实践早于这个名称：LangGraph、Microsoft Agent Framework 和 Google ADK 的官方文档分别称其为图编排或 graph-based workflow。参见 https://www.drjoshcsimmons.com/writing/we-are-entering-the-graph-engineering-phase、https://x.com/steipete/status/2078277297791189132、https://docs.langchain.com/oss/python/langgraph/overview、https://learn.microsoft.com/en-us/agent-framework/workflows/、https://adk.dev/workflows/。

[4] Anthropic. “Building effective agents”, 2024 年 12 月. https://www.anthropic.com/engineering/building-effective-agents

保持透明。明确显示 Agent 的规划步骤、执行日志和决策轨迹——这不只是为了调试方便，也是让用户建立信任的前提。因为黑箱里的错误一旦发生，外部观察者既无法定位也无法纠正。

设计好工具接口 (ACI，Agent-Computer Interface)。ACI 强调的是从 Agent 视角设计接口 (让 Agent 容易理解和使用)，而非传统 API 从程序员视角设计接口。工具的命名和参数要直观，容易误用的地方要从设计上让错误无法发生——比如 SIM 卡的缺角让卡片只能从一个方向插进卡槽，避免了用户插反的错误；微波炉门没关好就绝不加热，避免了用户开门加热的危险行为。这种“用设计消除错误”的思路，在制造业里有一个专门的术语，叫**防呆** (Poka-yoke)，源自丰田生产体系。设计不好的工具会让再强的模型也频繁出错——因为模型与工具之间唯一的沟通通道就是接口本身，模糊的接口会被模型放大成系统性的错误。

以下三节展开 Harness 工程中三个独立但重要的主题：模型选型、编排模式、护栏与安全性。它们都不属于 Harness 五要素本身，但都是工程实践中绕不开的决策。

1.2.3 如何选择模型

在讨论编排模式之前，先回答一个实操问题：应该选什么样的模型来驱动 Agent?

模型是 Agent 的智能基座，选对模型往往比优化提示词更有效。由于模型迭代极快，本节不推荐具体的模型版本，而是提供一些选型思路。

闭源模型。目前 Agent 开发中最常用的两大闭源模型厂商是 OpenAI (GPT/o 系列) 和 Anthropic (Claude 系列)。闭源模型通常在能力上领先，但成本较高且受限于厂商的 API 策略。选模型时不要只看排行榜，**要在你自己的任务上做评估**（见第七章）。

开源模型。在本书编写时，开源模型与闭源模型之间的差距在 6 个月以内，但成本显著低于闭源模型。如果你的业务场景对模型能力没有很高要求，开源模型是务实的选择。开源模型成本低、可私有化部署、支持微调定制，适合对成本敏感或有数据合规要求的场景。DeepSeek、Kimi、GLM 是国内 Agent 能力较强的模型。需要注意的是，不同模型在工具调用方面的能力差异很大，选型前务必在具体场景中测试。

能力之外，还要考虑模型的策略边界。模型在基准测试中具备某种能力，并不意味着承载它的产品一定允许用户调用这种能力。不同厂商会对网络安全、模型蒸馏、模型提取、隐私数据和高风险操作设置不同的策略边界；同一个任务在聊天产品、Coding Agent 和 API 中也可能得到不同结果。因此，模型选型不能只比较准确率、价格和速度，还要在自己的真实任务上测试：模型是否愿意执行、接口是否暴露所需能力，以及服务条款是否允许这种使用方式。对于业务关键任务，还应提前准备人工接管或其他合规模型作为替代路径。

绝大多数 Agent 需要支持思考 (Reasoning) 的模型。 Agent 需要进行多步思考、工具选择等复杂决策，不带思考能力的模型在这些任务上表现往往很差。只有极少数场景例外，例如只执行单步简单任务，或在 Computer Use 中仅需点击固定位置的简单 GUI 操作，此时不带思考的模型也能胜任。但只要涉及多步思考或动态决策，就一定要选择支持思考的模型。

关注输出速度和多模态能力。除了成本，还有两个容易被忽视的维度。一是**输出 token 的速度**：Agent 往往需要多轮推理，每轮都要等待模型输出完成才能执行下一步，所以输出速度直接决定了端到端的响应延迟——如果一个 Agent 任务需要 20 轮推理，每轮慢 2 秒就意味着总共多等 40 秒。二是**多模态支持**：如果你的 Agent 需要理解图片、音频或视频，多模态能力就是硬性要求，不同模型在这方面的差异很大。

1.2.4 编排模式：工作流与自主

编排模式是 Harness 中“上下文与工具”层面的组织方式——它决定了上下文如何在 LLM 调用之间流动、工具如何被调度，以及 Agent 的执行路径是预先设定还是动态生成。Agent 系统的编排方式经历了从简单到复杂的演进过程，每种模式都有其适用场景和相应的取舍。根据 Anthropic 与数十个团队合作构建 LLM Agent 的经验[5]，最成功的实现往往不是使用复杂的框架，而是采用简单、可组合的模式。

[5]Anthropic. “Building effective agents”, 2024 年 12 月. https://www.anthropic.com/engineering/building-effective-agents

在构建 LLM 应用时，应遵循“从简单到复杂”的原则：首先考虑单个 LLM 调用。如果通过优化提示词和上下文示例就能解决问题，就不要引入 Agent 系统。当需要多步骤处理时，对于可以清晰分解为固定子任务的场景，考虑使用工作流。只有当需要动态决策和灵活的执行路径时，才使用自主 Agent。需要记住的是：Agent 系统通常会用延迟和成本换取更好的任务性能，应该谨慎权衡这种交换是否值得。

一个常见的反面例子，是一开始就构建一个非常复杂的工作流或多 Agent 系统。例如为“从百万条聊天记录中提炼个人记忆”设计 Agent，一些 AI 模型很容易迅速画出一条看似严谨的流水线：先切分对话片段，再依次安排提取、证据核验、身份解析、记忆整理和合并复核 Agent，最后建立事实图、覆盖账本和不可变版本。每个部件单看都有道理，组合起来却是非常低效、不可靠的。因为复杂工作流的执行拓扑是固定的，遇到新的例外，系统很容易继续增加节点，架构越来越复杂，通用性却越来越差。模型原本可以根据上下文处理的语义判断，反而被工作流写死了。

因此，**Harness 很重要，不等于 Harness 越复杂越好。**Manus 官网用 “Less structure, more intelligence.” 概括这种取舍[6]：先给一个能力足够的 Agent 清晰的目标、必要的上下文和可组合的工具，用自然语言告诉它如何像人一样查证、比较、处理冲突；程序只固化权限、不得覆盖原始材料、原子发布等必须始终成立的边界。只有业务约束本身要求，或评估反复暴露出某类稳定失败时，才把对应步骤升级为专门的验证器、独立 Agent 或确定性流程。好的结构不是替 Agent 预演全部思考，而是守住边界，并把边界内的决策空间还给模型。

1.2.4.1 工作流模式：确定性的编排

工作流（Workflow）是通过预定义的代码路径来编排 LLM 和工具的系统。它的执行路径是确定性的，由开发者预先设计好——每一步做什么、下一步去哪里，都是代码写死的，LLM 只在每个节点内部负责理解和生成。

以一个订机票 Agent 为例，工作流可以设计为四个固定节点：

1. **核实用户身份**——调用身份验证 API，确认用户是谁
2. **搜索可用航班**——根据用户需求查询航班数据库
3. **完成付款**——调用支付接口扣款
4. **确认预订**——调用预订 API 锁定座位，向用户发送确认信息

每个节点内部可以使用 LLM（例如用自然语言理解用户的出行需求），但节点之间的流转顺序是代码固定的——系统不会在付款完成之前去预订座位，也不会在身份核实之前开始搜索航班。

工作流模式有两个核心优势。第一是**严格的流程控制**：开发者可以确保关键步骤不被跳过或乱序执行，例如“付款前不能预订”这类业务规则通过代码强制执行，不依赖 LLM 的判断。第二是**安全性**：由于执行路径是确定的，提示注入或模型犯错最多只能影响当前节点内部的处理，无法让 Agent 跳到不该执行的分支——攻击面被限制在单个节点内。

工作流的主要局限是**缺乏变通性。**当出现预设流程未覆盖的情况时（例如用户在付款环节临时想改签、或航班突然取消需要推荐替代方案），固定的节点路径无法灵活应对，只能走预设的异常处理分支或将控制权交还给人类。

举一个最简单的工作流例子：**文生图。**用户的需求往往就是一句大白话，比如“帮我画一个 AGI 实现以后程序员的工作场景”；可 Stable Diffusion 这类文生图模型只接受特定风格的提示词——逗号分隔的英文标签、质量词、负面提示词。所以工作流要在用户和生图模型之间安排两个固定节点：

1. **提示词改写**——用 LLM 把用户的自然语言需求改写成文生图模型习惯的提示词格式。对于上面的例子，“AGI 实现以后程序员的工作场景”是个很宽泛的需求，因此 LLM 还需要认真思考（例如“AGI 实现之后程序员不需要写代码了，因此应该画一个程序员在海边晒太阳，通过脑机接口指挥 AI 员工”），然后给出具体的场景描述。
2. **图片生成**——用改写后的提示词调用文生图模型，得到图片。

[6] Manus, “Less structure, more intelligence.” https://manus.im/

执行路径是代码写死的。这个工作流里的 LLM 节点做的是**翻译**，即把人话转成工具听得懂的输入格式，它存在的原因是文生图模型“听不懂人话”。这种专门给工具（或模型）的能力短板打补丁的 Harness 代码，不妨称为**适配层**。

但如果把生图工具换成具备**原生图像生成**能力的多模态模型，比如 Nano Banana 2、GPT-Image 2，就不再需要提示词改写了。不管用户怎么措辞，模型自己就能听懂、直接出图。

> **实验 1-4 ★：文生图工作流与原生图像生成的对照**
> 让同一句大白话需求走两条路线。**工作流路线**：LLM 先把需求改写成 Stable Diffusion 风格的提示词，再调文生图模型出图；**原生路线**：把这句话原样发给支持原生图像生成的多模态模型（如 GPT-Image 2），一次调用直接出图。
> 对照看：提示词改写节点把原始需求改成了什么样子，以及两条路线出的图谁更贴近原始需求。值得分两类需求对照：一类是描述具体的（比如指定了海报文案）；另一类是宽泛的（比如上面的 AGI 工作场景），这类需求下工作流路线仍可能有自己的优势。

这个实验说明：**Harness 里那些给模型能力短板打补丁的部分，会随着模型变强被模型自己内化。**仅仅在本书第一章中，这样的事就已经发生了好几轮：few-shot 示例、“让我们一步一步思考”之类的提示词技巧，被指令微调和推理模型内化了；输出格式修复、JSON 解析容错，被结构化输出和原生工具调用内化了；文生图的提示词改写，被模型的原生多模态理解与生成能力吃掉了。每一轮内化，消灭的都是“翻译”和“脚手架”这类适配层代码。

1.2.4.2 自主 Agent：动态自主决策

当工作流的固定路径无法满足需求时，我们就需要**自主 Agent**（Autonomous Agent）。自主 Agent 与工作流的核心区别在于：执行路径不是预先定义的，而是 Agent 根据**环境反馈**实时决定的。

仍以订机票为例：自主 Agent 不需要预定义四个固定节点。用户说“帮我订下周三去上海的机票”，Agent 会自行决定先搜索航班、发现需要登录、于是先核实身份、再回来搜索、发现最便宜的航班需要转机、主动询问用户是否接受、用户说不要转机、Agent 调整搜索条件……

这意味着自主 Agent 需要具备自主规划的能力——自主决定执行步骤，还需要能识别失败、调整策略，而不只是在出错时停下来。但自主性不等于无限制——必须设计明确的**停止条件**（任务完成、达到最大迭代次数或遭遇不可恢复的错误），否则 Agent 容易陷入死循环或过度执行。

从实现角度看，自主 Agent 本质上就是在一个循环中使用工具的 LLM，通过持续获取环境反馈来推进任务——这正是前面介绍的 ReAct 循环。常见的退出条件包括：调用最终输出工具、模型返回没有任何工具调用的响应，或者遇到错误、达到最大轮数。

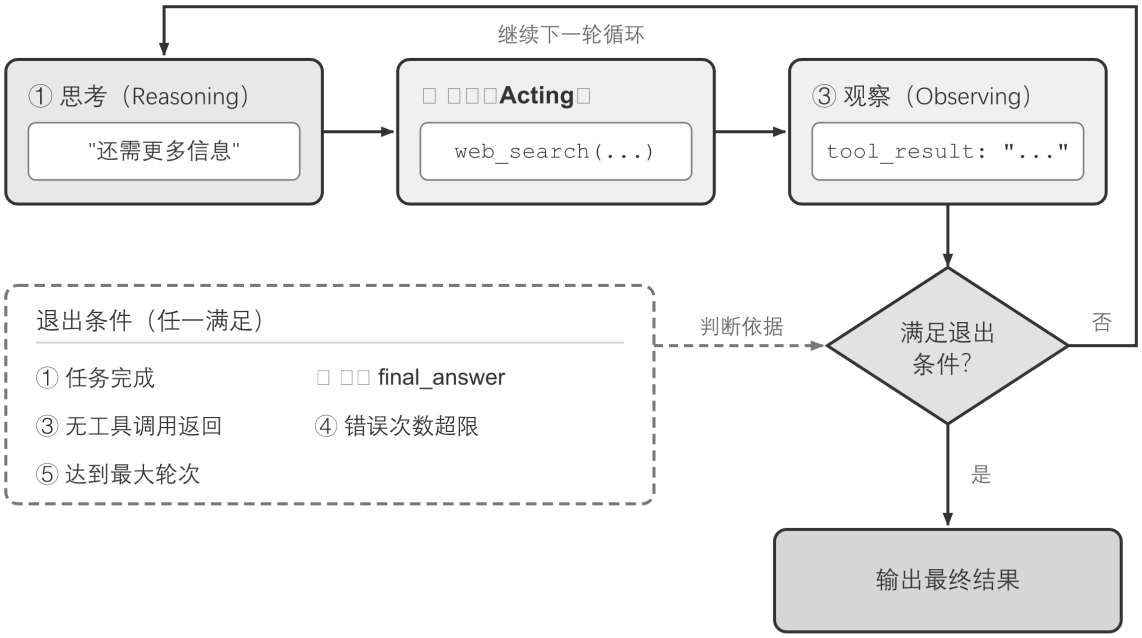


图 1-6 自主 Agent 的执行循环

自主 Agent 特别适用于开放式的问题——这类问题难以预测所需的步骤数量。典型的应用场景包括：Coding Agent 解决 SWE-bench（Software Engineering Benchmark，一个评估 Agent 自动修复真实 GitHub Issue 能力的基准测试）任务，"计算机使用"（Computer Use）Agent 像人类一样操作计算机界面，以及需要迭代搜索和分析的研究任务。

不过，自主性也带来了更高的成本和潜在的复合错误风险。因此在部署自主 Agent 时，必须在沙盒环境中进行充分的测试，设置适当的护栏和监控机制，并在关键决策点考虑加入人机协作的检查点。

1.2.4.3 两种模式的选择与混合

实践中，工作流和自主 Agent 并非非此即彼——很多系统会混合使用两种模式：关键的、有严格合规要求的流程用工作流来确保可靠性，需要灵活决策的部分切换到自主模式。例如，n8n 是一个成熟的工作流自动化开源框架，开发者通过可视化界面拖拽功能组件来构建 Agent，可以在同一个系统中同时使用工作流节点和自主 Agent 节点。

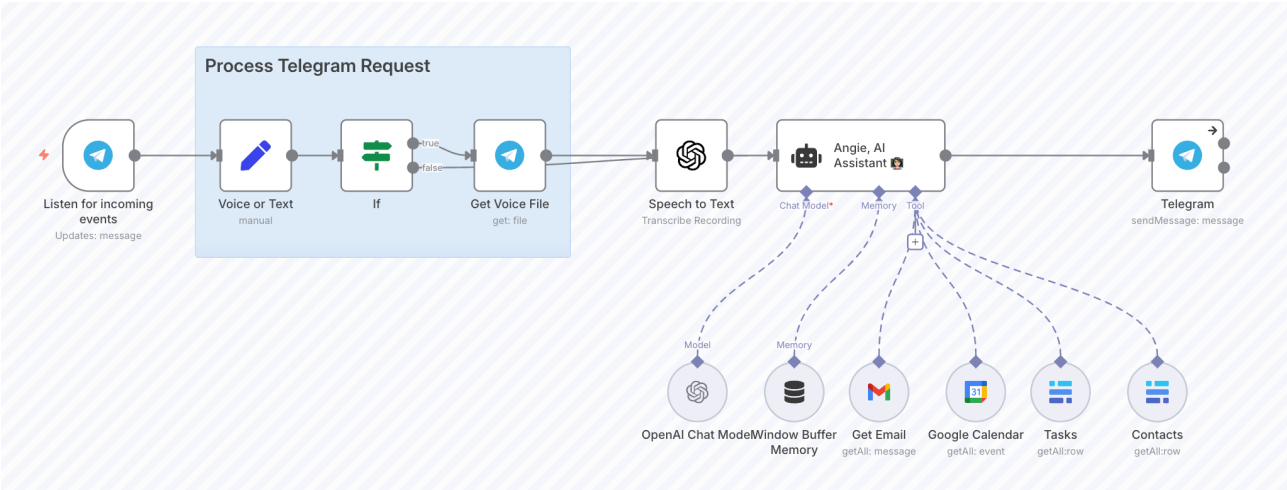


图 1-7 n8n 工作流编辑器界面

还有一种混合方式是**先由自主 Agent 把工作流写出来，再由工作流去执行。**Agent 读完任务后自行决定拓扑并生成一段编排代码；代码一旦生成，执行阶段就退回工作流的确定性。这样既保留了自主 Agent 面对未知任务的灵活性，又不必让模型在每一次调度上都参与决策。第十章会详细讨论这种形态。

1.2.4.4 主流 Agent 框架简要对比

下表梳理了当前主流的 Agent 框架/平台，帮助读者根据场景快速选型：

框架/平台	核心定位	编排模式	开发方式	适用场景
Codex Harness	Codex 的开源 Agent 运行时	自主	代码优先，可嵌入自有应用	Coding Agent、把 Agent 嵌进自家产品
Claude Agent SDK	生产级 Agent 开发框架	自主	代码优先	复杂自主任务、Coding Agent
LangChain / LangGraph	通用 LLM 应用框架	工作流 + 自主	代码优先	复杂链式思考、多步骤工作流
n8n	可视化工作流自动化	工作流 + 自主	低代码（可视化拖拽）	业务自动化、非技术团队
Dify	LLM 应用开发平台	工作流 + 对话式	低代码（可视化 + API）	企业级 RAG、知识库应用
CrewAI	角色化多 Agent 编排	Multi-Agent 协作	代码优先	团队式任务分解与执行
OpenClaw	开源全能个人 Agent	自主 + 事件驱动	配置 + 代码（自托管）	个人助理、Deep Research、Computer Use、多平台消息集成
DeepSeek Harness	Agent 自进化框架	一切皆插件	代码优先，方便定制	Agent 开发者、研究者
Pi	极简 Coding Agent 框架	自主	代码优先，方便定制	Agent 开发者

表中前两行值得单独澄清。Codex 是 OpenAI 的 Coding Agent 产品（App、CLI、IDE 扩展），Codex Harness 就是驱动这几种形态的那一层运行时[7]。Codex Harness 提供三条集成路径：`codex exec` 适合脚本与 CI 里的一次性任务；Codex SDK 适合第三方应用代码启动、恢复和流式处理任务；app-server 则通过 JSON-RPC 协议提供持久会话、事件流与审批回调，适合把 Agent 直接做进产品。Claude Agent SDK 与 Claude Code 也是类似的关系，区别在于 Claude 侧对外开放的是 SDK 接口，Harness 实现本身并不开源。

注意，Agent 框架发展迅速，在你阅读本书时，很可能有些框架已经过时，又有新的框架流行起来。因此，学会某个具体框架的用法并不重要。选择框架时，关键考量不在于框架本身的复杂度，而在于它能否用尽可能少的抽象层让你专注于业务逻辑。

编排模式解决的是上下文与工具的组织问题。但光能做事还不够，还要确保做得对、做得安全。为此，需要设置护栏，将约束、验证与纠正落实到实践中。

1.2.5 护栏与安全性

护栏（Guardrails）构成了保障 Agent 行为安全可控的分层防线，可用于管理数据隐私风险（例如防止系统提示泄露）或声誉风险（例如确保模型行为与品牌形象一致）。实践中可以先针对已识别的风险设置护栏，再在发现新漏洞时逐步添加。本节只做高层次的概览，

[7] OpenAI. "Codex as a platform: build on the open agent harness"，2026 年 8 月

确立全书统一的分层骨架；具体的实现细节将在第二章（上下文层：提示注入防护）、第四章（执行层：工具权限控制）和第五章（执行层与数据层：代码执行安全、信任边界下移）分别展开，初次阅读无需深究。

单个护栏不太可能提供足够的保护，多个专门的护栏组合使用，才能构建出更有韧性的 Agent 系统。

护栏也存在另一类失败：**误拒绝**。为了降低危险请求被放行的概率，模型可能同时拒绝一部分合法但形式敏感的任务，例如经过授权的安全测试、模型蒸馏研究。因此，护栏评估不能只测试“应当拒绝的请求是否被拦截”，还要测试“明确允许的请求是否能够正常完成”。

1.2.5.1 护栏类型

按防护位置可以分为三层：**上下文层、执行层、数据层**。这三层不是按请求处理的先后顺序排的，而是按**被绕过的难度**排的——越靠下的层越不依赖模型自己的判断，因此越难被一次成功的攻击穿透。本书后面所有的安全讨论都挂在这棵树上。

上下文层护栏管的是**模型能看到什么**，在内容进入上下文之前拦截，通常包含四种机制。**相关性分类器**标记偏离主题的查询，比如编程助手收到“帝国大厦有多高？”这类无关问题。**安全分类器**检测越狱（Jailbreak，即诱导模型绕过安全限制）和提示注入（Prompt Injection，即在输入中嵌入恶意指令），两者的关键区别在于：越狱是用户自己试图绕过模型的安全限制，提示注入则是攻击者通过外部数据（如网页内容、文档）间接操纵模型行为。**内容审核**标记有害或不当的输入，如暴力、歧视性内容。**基于规则的保护**则采用确定性措施，包括黑名单、输入长度限制、正则表达式过滤器，用以防范 SQL 注入等已知威胁。来源标注与“指令 / 数据”分离也属于这一层，第二章会展开。

上下文层护栏的一个代表性工业实践是 Anthropic 的 Constitutional Classifiers[8]。其核心机制有三点：一是**规则驱动**——用自然语言写成的规则（明确规定哪些内容允许、哪些禁止）生成合成训练数据，训练输入输出分类器；二是**上下文联合判断**——新一代系统把用户提问和模型回答放在一起检查，因为有些回答单独看毫无问题（如“如何使用食品调味料”），只有对照提问才能发现“食品调味料”其实是化学试剂的暗语；三是**两级筛查**——先用一个极轻量的探针（直接读取模型内部激活，几乎零成本）检查所有对话，发现可疑之处再交给更强的分类器复审，而不是直接拒绝。这样一来，第一级即使误报较多也不影响用户体验，成本也大大降低。

但这一层有一个结构性上限：**处在同一个上下文里的 Agent，很难判断自己是否已经被注入**。所以上下文层只能降低攻击成功率，给不出保证——这正是必须有下面两层的原因。

执行层护栏管的是**模型能做什么**，在动作真正生效之前验证。其核心是**工具风险评级**：根据操作是否可逆、权限等级、财务影响，为每个工具标注风险等级（低/中/高），高风险操作需额外审查或人工确认。关键在于这类复核必须由**上下文之外**的机制完成——独立的审查进程、最小权限凭证、沙盒隔离、人在回路——否则它会和被注入的 Agent 一起沦陷。返回给用户的回复本身也是一次动作（第四章把它归为用户沟通工具），因此**输出检查**同样属于这一层：**PII 过滤器**审查输出中的个人身份信息（如身份证号、手机号），防止不必要暴露；**输出验证**则通过内容检查确保回复与品牌价值一致。

数据层护栏管的是**世界最终能被改成什么样**，把“谁能对哪条数据做什么”交给一层稳定的、经过人类审查的机制强制执行：数据库的行级安全策略、约束与校验器、受控视图与存储过程，以及由受信任运行时绑定、无法被伪造的访问上下文。这一层的价值在于它不依赖上面两层是否正确——即使提示注入得手、生成的代码完全漏写了权限判断，越权操作仍会在数据层被拒绝。第五章会以动态生成软件为例展开这一层。

1.2.5.2 人工干预

人工干预（Human in the loop，又称人在回路）是一个关键的保护措施，它让 Agent 能够在不损害用户体验的情况下提升实际性能。这在部署早期尤为重要，有助于识别失败模式、发现边缘情况并建立健壮的评估周期。

实施人工干预机制，可以让 Agent 在无法完成任务时平稳地移交控制权。在客户服务中，这意味着将问题升级到人工客服；对于 Coding Agent，这意味着将控制权交还给开发者。

[8] Anthropic. “Next-generation Constitutional Classifiers: More efficient protection against universal jailbreaks”, 2026. https://www.anthropic.com/research/next-generation-constitutional-classifiers; 论文: Cunningham et al., “Constitutional Classifiers++: Efficient Production-Grade Defenses against Universal Jailbreaks”, arXiv:2601.04603

通常有两种主要情况会触发人工干预：

超过失败阈值为 Agent 的重试次数或操作次数设置上限。如果 Agent 超过了这些限制，就应该升级到人工干预。

高风险操作涉及敏感、不可逆或高风险的操作时，应触发人工监督，至少在团队对 Agent 可靠性建立起足够信心之前是如此。典型的例子包括授权大额退款或付款等。

回到 Harness 五要素的主线——下面看看它和本书的结构是什么关系。

1.2.6 Harness 五要素与「构建」部分的对应

先说清楚两个公式的关系，以免读者记住两套骨架。全书的结构骨架只有一个，就是引言和后记反复使用的 **Agent = LLM + 上下文 + 工具**：第二至六章讨论构建，第七至九章讨论评估与进化，第十章讨论协作。**Agent = Model + Harness** 不是与它并列的另一套划分，而是同一事物在生产形态下的展开——它把“上下文”和“工具”两项细分为上下文管理、工具接口、约束、验证、纠正五项职责，因此，它是**构建部分内部的一个观察视角**，而不是覆盖全书十章的目录。

在这个范围内，Harness 五要素与第二至五章有清晰的对应：

Harness 要素	对应章节	核心内容	安全关注点
上下文管理	第二章（上下文工程）	提示工程、Agent 状态栏、上下文压缩、Agent Skills	提示注入、上下文污染
上下文管理（跨会话）	第三章（用户记忆和知识库）	用户记忆、RAG、结构化索引、智能体化 RAG	敏感信息暴露、隐私保护
工具接口与约束	第四章（工具）	工具分类、权限控制、MCP 标准、主动工具发现	误操作、未授权访问、不可逆操作
验证与纠正	第五章（Coding Agent 与通用 Agent）	Coding Agent 的 Harness、测试驱动、代码化规则	身份冒用、责任归属

第六章（交互）不属于五要素中的任何一项，它扩展的是观察与动作空间本身的模态与时机；第七至九章讨论的是**怎么知道 Harness 建对了、以及怎么让它持续变好**；第十章则把单个 Agent 的 Harness 换成多个 Agent 的协作结构。把这些章也塞进五要素的格子里，只会让格子失去区分力。

安全同样不按章划分：它是贯穿全书的横切关注点（Cross-cutting Concern，即一个影响系统多个部分的问题），按前一节的三层护栏组织——上下文层、执行层、数据层。上表的”安全关注点”一列，给的是每一章在这三层里最主要的落点。

Anthropic 在构建长时运行 Agent 时的实践展示了 Harness 设计如何解决模型本身无法解决的问题。他们将复杂任务分解为”初始化 Agent”（设置环境、分解任务列表）和”执行 Agent”（在每个会话中增量推进并留下清晰的交接产物），通过结构化的 Harness 解决了 Agent 在长任务中”上下文耗尽”和”过早声明完成”的问题。后续章节将逐一深入 Harness 的各个组件——第二章从最核心的上下文工程开始，第五章将专门展开 Harness 工程在 Coding Agent 中的完整实践。

1.3 贯穿全书的设计模式

后面章节会反复用到同一批设计模式，因此在这里一次性命名并给出规范定义。

提议者—审核者（Proposer-Reviewer）：产出与评判由两个不共享上下文的角色分别承担，评判方看到的是产物本身——渲染结果、测试输出、结构化的调用参数——而不是产出方的推理过程。它成立的前提是**自审不可靠**：同一个上下文中的模型难以发现自己的认知盲区，也很难判断自己是否已被注入。第三章用它更新知识，第四章用它做工具调用的事前审批与事后验证（Sidecar 是它的一个只读变

体），第五章的 PPT、视频与日志三个实验都以它为骨架，第七章用它评估 UI，第九章用它审核更新提案，第十章讨论了它在对等协作中的形态，以及为什么不能让同一个 Agent 自审。

渐进式披露 (Progressive Disclosure)：不把全部信息一次性放进上下文，而是先给一份可检索的目录，再按需加载细节。它同时优化两件事——上下文预算与选择精度。第二章的 Agent Skills 是最典型的形态（元数据常驻、正文按需加载），第三章的分层检索、第四章的主动工具发现与分页截断、第十章的 Agent 发现都是它的变体。

只增不改 (Append-only)：状态以追加的方式演进，已经写下的内容不再回头修改。换来的是可缓存、可重放、可审计。第二章的 KV Cache 前缀稳定性是它的性能形态——改动越靠前，作废的缓存越多；第三章的事件式记忆、第四章把新工具的 schema 追加到轨迹末尾而不是插回前缀，都是同一条纪律。

边界集 + 保留集 (Boundary Set + Retention Set)：任何一次修改都要同时在"它应当改变的那批样本"和"它不应当影响的那批样本"上验证。只测前者会把过拟合当成进步，只测后者会把无效修改当成安全。第七章的回归任务、第八章的训练与评估隔离、第九章的更新提案验证都建立在这对集合上。

最小 diff + 可回滚：每次修改尽量小、带来源、可单独回滚，而不是整体重写。它让归因成为可能——出了问题能定位到具体哪一次改动。第三章的知识更新、第五章的代码补丁、第九章的 Prompt 与程序更新都遵循这条；本章开头给出的三条更新路径（上下文内适应、外部产物更新、参数更新），也正是按可回滚程度从高到低排列的。

1.4 本章小结

本章从实践出发，建立了理解和构建 AI Agent 的基础框架。

Agent = 大脑 + 眼睛 + 手脚：LLM 是大脑（决策核心），上下文是眼睛（决定它能看到什么），工具是手脚（决定它能做什么）。三者缺一不可。

扩展眼睛和手脚是最主要的能力杠杆：在模型固定时，重新定义或扩展观察空间与动作空间——也就是扩展上下文和工具——往往能直接把原本不可解的任务变为可解。Manus 和 OpenClaw 的演进都说明，通用性很大程度上来自接口边界的扩大；这种扩大必须按需进行，并配合权限控制和验证。

眼睛（上下文）是决定性的因素：上下文由静态前缀（系统提示词 + 工具定义）和动态轨迹（消息历史）构成。消融实验表明，各组件并不等价：去掉工具定义或工具执行结果会直接夺走行动与闭环能力，去掉另外两者的代价则取决于信息能否从当前观测重建。ReAct 循环的本质是通过不断追加轨迹来让模型持续推进任务。

Harness 是竞争力所在：模型能力正在商品化，真正的差异在于 Harness——围绕上下文和工具构建的约束、验证与纠正机制，确保 Agent "可靠地做事"。在生产级的 Agent 系统中，Harness 的绝大部分代码都在实现这些保障机制，而不仅仅是上下文和工具本身。

从工作流到自主 Agent：先优化提示词，再考虑工作流，最后才引入自主 Agent——这是降低意外风险最实用的顺序。每种编排模式都有其适用场景，不存在通用最优解。

五个设计模式贯穿全书：提议者—审核者、渐进式披露、只增不改、边界集 + 保留集、最小 diff + 可回滚。

安全是架构问题：安全问题从第一行代码就要考虑，而不是上线前打补丁。护栏按被绕过的难度分为上下文层、执行层与数据层三层，后续各章的安全讨论都挂在这个骨架上。

下一章将深入探讨 Harness 中最核心的组件——上下文工程。关于 Agent 概念在强化学习中的学术渊源，以及传统 RL 与现代 LLM Agent 的深入对比，我们将在第八章系统展开。

以下思考题旨在引导读者深入思考本章的核心概念，不设标准答案。

深度思考

思考题

1. ★★ 如果你只能给一个 Agent 系统增加一项能力——更强的模型、更丰富的上下文还是更多的工具——你会选哪个？在什么条件下你的选择会改变？
2. ★★★ ReAct 循环中，累计缓存读取量随轮数近似二次方增长。如何降低这种增长？
3. ★★ “模型即 Agent”范式意味着模型在工具调用决策上越来越自主。但本章论证了 Harness 工程的重要性反而在增加。这两个趋势如何共存？Agent 框架未来的核心价值体现在哪些方面？
4. ★★ 消融实验中“工具结果反馈”的缺失会让 Agent 反复重试直到耗尽迭代预算。在生产环境中，除了工具结果缺失，还有哪些情况可能让 Agent 陷入这种循环？你会设计怎样的检测和终止机制？
5. ★ 本章用感知、行动、策略三个维度分析了五个 Agent 产品。请选择一个你日常使用的 AI 产品，用这三个维度进行分析，并思考它的架构设计是否合理。如果由你来设计这个 AI 产品，有哪些改进空间？
6. ★★ 如果你要设计一个专门处理航班订票的客服系统，你会选择工作流模式还是自主 Agent 模式？有没有可能在同一个系统中混合使用两种模式？
7. ★★★ 护栏部分提到了工具风险评级。如果一个工具在大多数情况下是低风险的，但在特定参数组合下变为高风险（如 `delete_file` 删除普通文件 vs 删除系统文件），你会如何设计动态风险评估？
8. ★★ 本章的 Agent 产品表格中，所有 Agent 的动作空间都是“开放式”的。一个受限的动作空间（比如只能从预定义选项中选择）在什么场景下反而优于开放式？
9. ★★ 人工干预机制要求 Agent 能“优雅地移交控制”。但在实践中，用户可能不在线、响应很慢、或者给出模糊的指令。此时 Agent 应该怎么办？
10. ★★★ 引言指出“好的设计原则应该穿越模型的迭代周期”，但实现这些原则的具体工程手段可能会随模型能力进步而过时。试举一个这样的 Agent 工程手段，并说明理由。